

INNER JOIN y LEFT JOIN para conectar tablas

Todas las consultas que has escrito hasta ahora trabajan con una sola tabla. Pero las preguntas de negocio reales cruzan información entre varias. ¿Qué vendió Tienda Latam en Argentina durante el último trimestre, desglosado por categoría de producto? Para responder eso necesitas datos de Pedidos, Clientes, Países, Detalle de pedidos, Productos y Categorías al mismo tiempo.

Sin JOIN, tendrías que hacer varias consultas separadas y cruzar los resultados a mano, como si estuvieras en Excel. Con JOIN, es una sola consulta. Eso es lo que esta clase te da.

Qué es un JOIN

Un JOIN combina filas de dos tablas en una sola consulta, usando una columna que las relaciona. Esa columna ya la conoces: es la clave foránea. El **ON** dentro del JOIN es donde le dices al motor cuáles son las columnas que conectan las dos tablas.

Hay dos tipos que vas a usar en el 90% de los casos:

- **INNER JOIN:** devuelve solo las filas que tienen coincidencia en las dos tablas.
- **LEFT JOIN:** devuelve todas las filas de la tabla izquierda, tenga o no coincidencia en la derecha.

Alias de tabla: imprescindible en cualquier JOIN

Cuando combinas dos tablas, ambas pueden tener columnas con el mismo nombre. Por ejemplo, **pedidos** y **clientes** tienen las dos una columna **cliente_id**. Para evitar ambigüedad, se le da un alias a cada tabla y se usa ese alias para especificar de dónde viene cada columna.

```
SQL
FROM pedidos AS p

INNER JOIN clientes AS c ON p.cliente_id = c.cliente_id
```

`p` es el alias de `pedidos` y `c` es el alias de `clientes`. A partir de ese momento, `p.cliente_id` viene de la tabla de pedidos y `c.nombre` viene de la tabla de clientes. Es más corto, más claro y evita errores.

INNER JOIN: solo lo que coincide en las dos tablas

Imagina que tienes el resultado de la clase anterior: los clientes con más pedidos, pero solo con su `cliente_id`. Alguien pregunta: "¿Y quién es ese cliente?" Para saberlo, tienes que traer el nombre desde la tabla Clientes. Eso es un INNER JOIN.

```
SQL
SELECT
    p.cliente_id,
    c.nombre      AS nombre_cliente,
    c.apellido    AS apellido_cliente,
    COUNT(*)      AS cantidad_pedidos
FROM pedidos AS p
INNER JOIN clientes AS c ON p.cliente_id = c.cliente_id
GROUP BY p.cliente_id, c.nombre, c.apellido
HAVING COUNT(*) >= 3
ORDER BY cantidad_pedidos DESC;
```

Tres cosas importantes en esta consulta:

El ON define la conexión. `p.cliente_id = c.cliente_id` le dice al motor: "une estas dos tablas donde el cliente sea el mismo". Sin este **ON**, el motor no sabe cómo relacionarlas.

El GROUP BY sigue la misma regla de siempre. Todo lo que aparece en el SELECT y no es una función de agregación tiene que estar en el GROUP BY. `c.nombre` y `c.apellido` no son agregaciones, así que van ahí aunque vengan de otra tabla.

INNER JOIN solo devuelve coincidencias. Si hay un pedido con un `cliente_id` que no existe en la tabla Clientes (lo que no debería ocurrir gracias a las claves foráneas, pero podría pasar en datos sucios), ese pedido no aparece en el resultado.

LEFT JOIN: todo lo de la izquierda, con o sin coincidencia

El INNER JOIN es perfecto cuando quieres cruzar información que existe en los dos lados. Pero a veces la pregunta es exactamente al revés: ¿qué hay en una tabla que no tiene nada en la otra?

Caso concreto: ¿qué clientes nunca han hecho un pedido? Esos clientes existen en la tabla Clientes, pero no tienen ninguna fila en la tabla Pedidos. Un INNER JOIN los descartaría porque no hay coincidencia. Un LEFT JOIN los conserva.

```
SQL
SELECT
    c.nombre,
    c.correo,
    p.pedido_id
FROM clientes AS c
LEFT JOIN pedidos AS p ON c.cliente_id = p.cliente_id
WHERE p.pedido_id IS NULL
ORDER BY c.nombre;
```

Lo que hace esta consulta:

1. **LEFT JOIN** mantiene todos los clientes, tengan o no pedidos.
2. Para los clientes sin pedidos, las columnas de la tabla Pedidos aparecen como **NULL**.
3. **WHERE p.pedido_id IS NULL** filtra solo los clientes donde no hubo ninguna coincidencia, es decir, los que nunca compraron.

El resultado es exactamente la lista de clientes sin pedidos: candidatos perfectos para una campaña de reactivación.

La diferencia visual entre los dos

Piénsalo con un ejemplo simple. Tienes diez clientes y ocho de ellos tienen al menos un pedido. Los otros dos nunca compraron.

Tipo de JOIN	Cuántos clientes devuelve	Los que nunca compraron
INNER JOIN	8	No aparecen
LEFT JOIN	10	Aparecen con NULL

La tabla izquierda en un LEFT JOIN es siempre la que aparece después del **FROM**. Todos sus registros se preservan en el resultado, sin excepción.

Cómo se construye un JOIN: la estructura completa

```
SQL
SELECT
    alias1.columna,
    alias2.columna,
    COUNT(*) AS agregacion
FROM tabla1 AS alias1
INNER JOIN tabla2 AS alias2 ON alias1.clave = alias2.clave
WHERE condicion_de_filas
GROUP BY alias1.columna, alias2.columna
HAVING condicion_de_grupos
ORDER BY criterio;
```

El JOIN va entre el **FROM** y el **WHERE**. Puedes encadenar varios JOINS seguidos para cruzar más de dos tablas, y cada uno tiene su propio **ON**.

Los errores más comunes al escribir JOINS

Olvidar el ON. Sin la condición de unión, el motor combina cada fila de una tabla con cada fila de la otra. Si una tabla tiene 1.000 filas y la otra tiene 2.000, el resultado tiene 2.000.000 de filas. Ese error se llama producto cartesiano y puede congelar una consulta.

No poner alias cuando las dos tablas tienen columnas con el mismo nombre. Si escribes `cliente_id` sin especificar `p.cliente_id` o `c.cliente_id`, el motor no sabe de cuál tabla tomarlo y devuelve un error de ambigüedad.

Poner columnas del JOIN en el GROUP BY con el alias equivocado. Si defines `pedidos AS p` y luego escribes `pedidos.cliente_id` en el GROUP BY en lugar de `p.cliente_id`, el motor no lo reconoce.

Con INNER JOIN y LEFT JOIN tienes las herramientas para cruzar cualquier par de tablas de Tienda Latam. El siguiente paso es aprender a combinar más de dos tablas en una sola consulta, que es donde las preguntas de negocio más complejas empiezan a tener respuesta.